

AI Reference

1. AI Tool(s) Used

- **Claude Code** (Opus 4.7)
 - **Antigravity** — 根據 [2026\(Spring\)_SS-Midterm Project.pdf](#) 規格轉為 [implementation_plan.md](#), 再轉為 [CLAUDE_PLAN.md](#) 供 Claude Code 逐 phase 執行
-

2. Scope of Usage

按開發階段 (phase) 分節。每節包含: **Prompt** → **做了什麼** → **修改的檔案** → **關鍵程式碼** → **簡要說明 / 微調**。

Phase 0 — 專案初始化

Prompt: 執行 CLAUDE_PLAN.md 的 Phase 0, 依 [事先準備.txt](#) 填入環境變數。

做了什麼: 用 `npm create vite@latest` 建 React 19 + Vite 8 骨架, 加入 Firebase / React Router / DOMPurify 三個依賴, 建立深色主題與工具函式。

修改的檔案:

- `package.json` — 加入 `firebase / react-router-dom / dompurify`
- `src/config/firebase.js` — Firebase 初始化、`export auth / db / googleProvider`
- `src/index.css` — 深色主題 CSS Variables、reset
- `src/utils/{constants,formatTime,sanitize}.js` — 共用工具
- `.env` — 12 個 `VITE_*` 變數

關鍵程式碼 — `src/config/firebase.js`:

```
const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
};
const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const googleProvider = new GoogleAuthProvider();
export const db = getFirestore(app);
```

簡要說明 / 微調:

- 偏離原計畫：Firebase Storage 需 Blaze 付費方案 → 改用 ImgBB 免費 API (`uploadProfileImage` / `uploadMessageImage`)，介面不變。

Phase 1 — 會員系統 (Email + Google)

Prompt: 進 Phase 1，實作 Email/Google 登入註冊。

做了什麼: 寫了 `AuthContext` 用 `onAuthStateChanged` 同步 `currentUser`，並訂閱 `users/{uid}` 文件即時取得 profile；建立兩個路由守門元件。

修改的檔案:

- `src/contexts/AuthContext.jsx` — 同步 auth + Firestore profile
- `src/services/authService.js` — register / login / logout + 中文錯誤訊息
- `src/pages/{LoginPage, RegisterPage}.jsx`、`auth.css`
- `src/App.jsx` — `PrivateRoute` / `PublicOnlyRoute`

關鍵程式碼 — `src/App.jsx` 路由守門:

```
function PrivateRoute({ children }) {
  const { currentUser, loading } = useAuth();
  if (loading) return <LoadingSpinner fullscreen />;
  return currentUser ? children : <Navigate to="/login" replace />;
}

function PublicOnlyRoute({ children }) {
  const { currentUser, loading } = useAuth();
  if (loading) return <LoadingSpinner fullscreen />;
  return currentUser ? <Navigate to="/" replace /> : children;
}
```

簡要說明 / 微調:

- `AuthContext` 多訂閱一層 `userProfile`: 原計畫只給 `currentUser`，但下游元件常要 `username` / `photoURL`，提早 Firestore 即時化省去重複 `getDoc`。
- 多寫 `PublicOnlyRoute`: 避免使用者已登入卻看到登入表單。

Phase 2 — 聊天室核心 (即時訊息)

Prompt: 進 Phase 2 實作私聊 / 群聊建立、Sidebar、ChatArea、即時訊息收發。

做了什麼: 用 Firestore `onSnapshot` 訂閱聊天室與訊息子集合；訊息發送時把 `senderName` / `senderPhoto` 一起 denormalize 寫入。

修改的檔案:

- `src/services/{chatroomService, messageService, userService}.js`
- `src/hooks/{useChatrooms, useMessages}.js` — 包 `onSnapshot` 生命週期

- `src/components/sidebar/*` — Sidebar / ChatroomList / CreateRoomModal / UserPicker
- `src/components/chat/*` — ChatArea / MessageList / MessageBubble / MessageInput

關鍵程式碼 — `src/hooks/useMessages.js` (onSnapshot pattern) :

```
useEffect(() => {
  if (!chatroomId) return undefined;
  const q = query(
    collection(db, 'chatrooms', chatroomId, 'messages'),
    orderBy('createdAt', 'asc'),
  );
  const unsub = onSnapshot(q, (snap) => {
    setMessages(snap.docs.map((d) => ({ id: d.id, ...d.data() })));
    setLoading(false);
  });
  return unsub;
}, [chatroomId]);
```

簡要說明 / 微調:

- **私聊去重:** 建私聊前先 `findPrivateChatroom` 查是否已存在, 避免兩人各建一間。
- **bug fix (同 Phase) :** `subscribeToChatrooms` 原本用 `where + orderBy` 需建 composite index, 未建好時 sidebar 變空白; 改成 `query` 只留 `where`、前端再排序, 並把 error 一路傳到 UI 顯示。

Phase 3 — 個人資料 + ImgBB 頭像上傳

Prompt: 進 Phase 3, 建立 ImgBB 上傳與個人資料編輯 Modal。

做了什麼: 用 `URL.createObjectURL` 做頭像 preview, 儲存時才上傳 ImgBB; 同步寫 Firebase Auth `displayName / photoURL`。

修改的檔案:

- `src/services/imgbbService.js` — 上傳前擋非圖片 + > 32 MB
- `src/components/profile/ProfileModal.jsx + .css`
- `src/contexts/UsersContext.jsx` — 訂閱整個 users collection 並 memo `usersById`

關鍵程式碼 — `ProfileModal.jsx` 儲存邏輯:

```
const handleSave = async () => {
  let photoURL = userProfile?.photoURL || '';
  if (pendingFile) {
    photoURL = await uploadProfileImage(currentUser.uid, pendingFile);
  }
  await updateUserProfile(currentUser.uid, { username, phone, address, photoURL });
  await updateProfile(auth.currentUser, { displayName: username, photoURL });
};
```

```
onClose?.();
};
```

簡要說明 / 微調：

- **登出搬進 ProfileModal**：原本 sidebar 同時放登出按鈕容易誤按，搬進 Modal + 紅色 ghost 按鈕，與 Telegram / Messenger 一致。
- **bug fix**：私聊 `chatroom.name` 原本寫死「建立者視角下的對方」，導致非建立者看到自己的名字。新增 `UsersContext` 動態 resolve 「members 中非自己的 uid → `usersById[uid].username`」。

Phase 4 — 訊息操作（編輯 / 收回 / 搜尋 / 圖片）

Prompt：進 Phase 4，實作訊息收回、編輯、搜尋、傳送圖片。

做了什麼：MessageBubble hover 顯示 … 選單，編輯走 `editMessage` 服務、收回前 `window.confirm`；MessageSearch 浮動 panel + `scrollIntoView` 跳轉高亮。

修改的檔案：

- `src/components/chat/MessageBubble.jsx + .css` — … 選單 + `is-highlighted` 紫光量
- `src/components/chat/MessageInput.jsx` — `editingMessage` prop、🗣️ 附加圖片
- `src/components/chat/MessageSearch.jsx + .css` — 浮動搜尋 panel
- `src/components/chat/ImagePreview.jsx` — 全螢幕圖片預覽
- `src/components/chat/ChatArea.jsx` — 新增 4 個 state（editing / search / highlight / preview）

關鍵程式碼 — `MessageBubble.jsx` 收回前確認：

```
const handleUnsend = () => {
  setMenuOpen(false);
  if (window.confirm('確定要收回這則訊息嗎? ')) {
    onUnsend?.(message);
  }
};
```

簡要說明 / 微調：

- **State 全部 lift 到 ChatArea**：edit / search / highlight 跨多元件，集中管理才能正確協調（例如收回正在編輯的訊息要同時清掉 editing state）。
- **bug fix（同 Phase）** — 下拉選單按不到：原本 `position: absolute` 被 `.message-list { overflow-y: auto }` 切掉，改用 `createPortal` 把選單搬到 `document.body` + `position: fixed`，並用 `useLayoutEffect` 算 trigger 的 `getBoundingClientRect()` 定位。

Phase 5 — 通知 + CSS 動畫整理 + XSS 防禦

Prompt：進 Phase 5，實作 Chrome Notification、整理 CSS Animation

做了什麼： 用瀏覽器原生 Notification API + Firestore onSnapshot 達成「不在當前聊天室或失焦時收通知」；DOMPurify 處理使用者可控字串。

修改的檔案：

- `src/services/notificationService.js` — `requestNotificationPermission / showNotification`
- `src/hooks/useNotification.js` — 訂閱所有 chatroom 的 messages, 去重 + 條件過濾
- `src/components/sidebar/NotificationButton.jsx` — 鈴鐺按鈕
- `src/components/common/NotificationToast.jsx` — 應用內 toast fallback

關鍵程式碼 — `useNotification.js` 去重 + 條件邏輯：

```
// onSnapshot 重新訂閱會把現有訊息當 added 全部回放, 需去重
const notifiedIdsRef = useRef(new Set());

snap.docChanges().forEach((change) => {
  if (change.type !== 'added') return;
  const msg = { id: change.doc.id, ...change.doc.data() };
  if (notifiedIdsRef.current.has(msg.id)) return;
  notifiedIdsRef.current.add(msg.id);
  // 自己 / 收回 / 系統訊息不彈
  if (msg.senderId === currentUserId) return;
  if (msg.isUnsent || msg.type === MESSAGE_TYPES.SYSTEM) return;
  // 在當前聊天室且頁面有焦點 → 不彈
  if (room.id === activeRef.current && document.hasFocus()) return;
  onNotifyRef.current?.(msg, room);
});
```

簡要說明 / 微調：

- **不用 FCM：** 在 Phase 0 就決定省掉 VAPID key。原生 Notification API 已足夠「使用者開著瀏覽器但切到別的視窗」場景。
- **bug fix (同 Phase) — 系統通知靜默失敗：** 原本通知請求寫在 `useEffect` 載入時就呼叫 → Chrome 政策要求必須由「使用者明確互動」觸發。新增 `NotificationButton` 點擊才呼叫；同時加 toast fallback, permission 沒給也仍能在 app 內看到。

Phase 6 — 進階功能 (AI Chatbot / Block / GIF / Reactions)

Prompt： 進行 Phase 6, 依序做 (6.1) Gemini AI 助手、(6.2) 封鎖使用者、(6.3) GIPHY GIF、(6.4) Emoji

6.1 — Gemini AI Chatbot

做了什麼： 自己寫 REST 呼叫 Gemini `generateContent` endpoint, 多輪對話只存 React state。

修改的檔案：

- `src/services/geminiService.js`
- `src/components/chatbot/ChatbotModal.jsx + .css`

關鍵程式碼 — chatWithGemini 對話歷史轉 Gemini 格式：

```
export async function chatWithGemini(userMessage, history = []) {
  // Gemini 要求 role 只能是 'user' / 'model', 本地用 'ai' 要轉
  const contents = [
    ...history.map((m) => ({
      role: m.role === 'ai' ? 'model' : 'user',
      parts: [{ text: m.text }],
    })),
    { role: 'user', parts: [{ text: userMessage }] },
  ];
  const res = await fetch(`${API_BASE}?key=${API_KEY}`, {
    method: 'POST',
    body: JSON.stringify({ contents, generationConfig, systemInstruction }),
  });
  // ...
}
```

簡要說明：不用 SDK 是因為自己寫只 ~50 行 + 對參數更透明；只存 React state 不寫 Firestore（依需求）。

6.2 — 封鎖使用者

做了什麼：用 `arrayUnion` / `arrayRemove` 改 `users/{uid}.blockedUsers`，雙向偵測（我封鎖他 + 他封鎖我）。

關鍵程式碼 — ChatArea.jsx 群聊雙向過濾：

```
const visibleMessages = useMemo(() => messages.filter((m) => {
  if (m.senderId === currentUser.uid) return true;
  const myBlocked = userProfile?.blockedUsers || [];
  if (myBlocked.includes(m.senderId)) return false;
  const theyBlockedMe =
    usersById[m.senderId]?.blockedUsers?.includes(currentUser.uid);
  if (theyBlockedMe) return false;
  return true;
}), [messages, userProfile, usersById, currentUser]);
```

簡要說明：私聊封鎖時 disable input + 顯示紅色 banner（不藏歷史）；群聊則用 `visibleMessages` 雙向過濾。

6.3 — GIPHY GIF Picker

做了什麼：MessageInput 加 GIF 按鈕，popover 內 search input + 2 欄縮圖 grid，搜尋 350ms debounce。

關鍵程式碼 — GifPicker.jsx debounce：

```
useEffect(() => {
  if (!query.trim()) {
```

```

    fetchTrending();
    return;
  }
  const t = setTimeout(() => fetchSearch(query), 350);
  return () => clearTimeout(t);
}, [query]);

```

簡要說明： GIPHY URL 是 CDN 直接可用，不過 ImgBB 中轉；訊息存 GIPHY URL 本身。

6.4 — Emoji Reactions

做了什麼： 訊息 hover 出現 ☺ 觸發 8 個 emoji 橫條，點擊 toggle；data shape 用 `emojis: { '👍': [uid1, uid2] }`。

關鍵程式碼 — `messageService.js` 用 dotted field path：

```

export async function addReaction(chatroomId, messageId, emoji, userId) {
  await updateDoc(messageDoc(chatroomId, messageId), {
    [`emojis.${emoji}`]: arrayUnion(userId),
  });
}

```

簡要說明： dotted field path 讓不同 emoji 互相獨立，新增 emoji 不需先讀取舊資料。

Phase 7 — RWD 響應式設計

Prompt： 進 Phase 7，把所有元件在 $\leq 768\text{px}$ / $\leq 480\text{px}$ 螢幕都能用，所有按鈕 / picker / modal / bubble 不能溢出或被擠扁。

做了什麼： 純 CSS @media 改寫，行為 (hamburger toggle、resize listener) Phase 0 已預埋。

修改的檔案：

- 13 個 CSS 檔案加 ≤ 768 / ≤ 480 規則
- 涵蓋 ChatPage / MessageInput / MessageBubble / GifPicker / EmojiPicker / Modal / Sidebar / auth / etc.

關鍵程式碼 — `auth.css` iOS 防 auto-zoom：

```

@media (max-width: 480px) {
  .auth-field input {
    font-size: 16px; /* 必須 ≥16px, 否則 iOS Safari focus 會自動放大整頁 */
    padding: 11px 12px;
  }
}

```

簡要說明 / 微調：

- **iOS auto-zoom trap**: input `font-size < 16px` 時 iOS Safari focus 會強制放大整頁，桌面用 15px / mobile 用 16px。
- **不動 JSX**: 避免影響事件流，純 CSS @media 解決。

Phase 8 — UI a11y polish

Prompt: 進 Phase 8, 按 CLAUDE_PLAN 的 checklist 把全域 polish 一輪: CSS variable、hover/focus、Loading、Modal、空狀態、cursor、form focus。

修改的檔案:

- `src/index.css` — 全域規則

關鍵程式碼:

```
*:focus-visible {
  outline: none;
  box-shadow: var(--focus-ring);
}

button:disabled {
  cursor: not-allowed;
}

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

簡要說明:

- **:focus-visible 而非 :focus**: 滑鼠點按不會殘留 ring，只有鍵盤 Tab 才亮。
- **0.01ms 不能用 0**: spec 規定 0 等於沒動畫，會讓某些 onAnimationEnd handler 不 fire。

Phase 8.5 — 文字顯示 bug 修復 + 未讀計數

Prompt:

1. 「請幫我改成我輸入甚麼內容 輸出就會是甚麼內容」— 想要 `<script>alert("example");</script>`、`<h1>example</h1>`、< 都「原文照顯示」，目前送出後 < 會變成 `<`。
2. 「想要新增還尚未讀訊息的聊天室 可以用特殊的符號或其他方式標記 並且可以顯示未讀幾則訊息」。

做了什麼: 把 `sanitizeInput` 改 identity (React 文字渲染本身就 XSS 安全) ; `sendMessage` 加 fanout 寫入每位 member 的 `unreadCounts`，進房間自動 markRead。

修改的檔案:

- `src/utils/sanitize.js` — identity 函式
- `src/services/messageService.js` — fanout + `markChatroomRead`
- `src/pages/ChatPage.jsx` — 進房間 effect 觸發 `markRead`
- `src/components/sidebar/ChatroomItem.jsx + .css` — 紫色未讀 badge

關鍵程式碼 — `sanitize.js`:

```
// React 的 {value} 文字渲染本身就會以 text node 輸出 (不會解析成 HTML) ,
// 所以 <script> 之類的内容只會「顯示為文字」、不會執行。
export const sanitizeInput = (input) => input;
```

關鍵程式碼 — `messageService.js` unread fanout:

```
const room = await getDoc(chatroomDoc(chatroomId));
const members = room.data()?.members || [];
const updates = { lastMessage: ..., lastMessageAt: serverTimestamp() };
members.forEach((uid) => {
  if (uid === payload.senderId) return; // sender 不加自己
  updates[`unreadCounts.${uid}`] = increment(1); // dotted field path
});
await updateDoc(chatroomDoc(chatroomId), updates);
```

簡要說明 / 微調:

- **為何 `sanitize` 變 `identity` 是安全的:** `grep` 確認無 `dangerouslySetInnerHTML`, 所有訊息都走 `{value}` JSX 文字內插。原本 `DOMPurify` 把 `<` 轉成 `<`, 被 `React` 再次原樣輸出 → 看到 `<` 字面, 這就是雙重轉義 bug。
- **dotted field path:** `updates['unreadCounts.${uid}'] = increment(1)` 可以原子地遞增 `map` 內的某個欄位, 舊文件無此欄會自動建立。

Phase 8.6 — 鈴鐺通知改 on/off toggle

Prompt: 鈴鐺按鈕改成可切換開 / 關。

做了什麼: 新增 `useNotificationMuted` hook (`localStorage` 持久化 + module-level listener `Set` 實現 cross-component 同步), 同時擋 `toast` 與瀏覽器通知。

修改的檔案:

- `src/hooks/useNotificationMuted.js` — 新檔
- `src/hooks/useNotification.js` — 用 `mutedRef` 同步最新值
- `src/components/sidebar/NotificationButton.jsx` — 重寫為 toggle

關鍵程式碼 — `useNotificationMuted.js`:

```

const STORAGE_KEY = 'notification_muted';
const listeners = new Set();

function read() {
  try { return window.localStorage.getItem(STORAGE_KEY) === '1'; }
  catch { return false; }
}

function write(value) {
  try { window.localStorage.setItem(STORAGE_KEY, value ? '1' : '0'); } catch {}
  listeners.forEach((fn) => fn(value)); // 通知所有訂閱者
}

export function useNotificationMuted() {
  const [muted, setMutedState] = useState(read);
  useEffect(() => {
    const fn = (v) => setMutedState(v);
    listeners.add(fn);
    return () => { listeners.delete(fn); };
  }, []);
  return [muted, (v) => write(Boolean(v)), () => write(!read())];
}

```

簡要說明 / 微調：

- **不用 Context, 用 module-level Set:** 只兩個消費端 (按鈕 + hook) , 多包一層 Provider 是 over-engineering。
- **mutedRef 必要:** useNotification 內 `onSnapshot` callback 在第一次 run 時就 closure 住當時的 `muted` 值, 後面再變不會反映; 用 `ref` + `useEffect` 同步才能在 callback 觸發當下讀到最新值。

Phase 8.7 — 離開群組 + 系統訊息

Prompt: 請幫我新增離開群組的功能。離開後左側群組聊天室會自動消失 但是其他不影響其他還在群組的人 只會有某人離開群組的訊息跳出

做了什麼: 用 `writeBatch` 原子寫入「移除成員 + 系統訊息 + 更新 `lastMessage`」三件事。

修改的檔案：

- `src/services/chatroomService.js` — 新 export `leaveGroup`
- `src/components/chat/MessageBubble.jsx` — SYSTEM type early return
- `src/components/chat/ChatroomInfoModal.jsx` — 新增「 離開群組」按鈕
- `src/pages/ChatPage.jsx` — selected room 從 list 消失時自動清空 selection

關鍵程式碼 — `chatroomService.js`：

```

export async function leaveGroup(chatroomId, userId, displayName) {
  const roomRef = doc(db, 'chatrooms', chatroomId);
  const sysRef = doc(collection(db, 'chatrooms', chatroomId, 'messages'));

```

```

const sysContent = `${displayName || '某位成員'} 已離開群組`;

const batch = writeBatch(db);
batch.set(sysRef, {
  type: MESSAGE_TYPES.SYSTEM,
  content: sysContent,
  senderId: userId,
  senderName: displayName || '',
  isEdited: false,
  isUnsent: false,
  emojis: {},
  createdAt: serverTimestamp(),
  updatedAt: serverTimestamp(),
});
batch.update(roomRef, {
  members: arrayRemove(userId),
  lastMessage: sysContent,
  lastMessageAt: serverTimestamp(),
});
await batch.commit(); // 原子操作：兩個都成功或兩個都失敗
}

```

簡要說明 / 微調：

- **為何用 writeBatch**：兩次獨立 await 在中途斷網會留下「成員已移除但無系統訊息」不一致狀態。batch 是 server-side atomic。
- **不走 sendMessage**：sendMessage 內含 unread fanout 對所有人 +1，離開群組是 housekeeping 訊息不應該推未讀數。
- **selected room 自動清空**：ChatPage 加 `useEffect: if (selectedChatroomId && !selectedChatroom) setSelectedChatroomId(null)`，將來「踢出 / 解散」也通用。

Phase 8.8 + 8.9 — Splash 開場 3D 動畫 (CodePen 風格碎片重組)

Prompt: Use CSS animation (2%) – Button hover is not an animation! 幫我在點開啟此 chatroom 一開始 幫我加入 logo 的動畫載入動畫 並且在其他適合的地方幫我加入好看的動畫特效 2D 3D 皆可

你可以幫我參考這個網站做的 css animation 幫我在開啟 app 的片頭改編成 幫我製作出屬於 chatroom 這個版本的 3D animation 用 2s → 後續：「動畫有點卡卡的 請幫我弄到順暢的動畫 時間不一定要 2s 依照你認為最適合的秒數 但是動畫要視順暢的」。

(8.8) 加 logo 載入動畫 + 其他適合的 CSS 動畫。(8.9) 改編 [CodePen 3D shatter](#) 變成 chatroom 版 → 「動畫有點卡卡的，請弄到順暢」。

做了什麼：用純 CSS (不引 Three.js) 模擬 64 → 36 個 shard 從 3D 空間飛回拼成 logo；後續為了順暢度大幅優化。

修改的檔案：

- `src/components/common/SplashScreen.jsx` — 新檔，36 shards `clip-path` 切割 favicon
- `src/components/common/SplashScreen.css` — `shardAssemble` keyframe 用 CSS custom properties

- `src/App.jsx` — `<SplashScreen />` 掛在 `routes` 旁
- 額外: `auth.css` / `ChatArea.css` / `ChatroomItem.css` 加 `shimmer` / `float` / `pulse` 動畫

關鍵程式碼 — `SplashScreen.jsx` shard 隨機起點 + delay:

```
function buildShards() {
  const shards = [];
  for (let r = 0; r < GRID; r++) {
    for (let c = 0; c < GRID; c++) {
      // 從中心向外的角度 + jitter
      const angle = Math.atan2(r - 2.5, c - 2.5) + (Math.random() - 0.5) * 0.7;
      const dist = 220 + Math.random() * 220;
      const tx = Math.cos(angle) * dist;
      const ty = Math.sin(angle) * dist;
      // 角落先到、中心後到 (diagonal-wave)
      const distFromCenter = Math.hypot(r - 2.5, c - 2.5);
      const delay = (1 - distFromCenter / Math.hypot(3, 3)) * 600;
      // clip-path 切出這格
      const cellSize = 100 / GRID;
      const clipPath = `polygon(${c*cellSize}% ${r*cellSize}%, ...)`;
      shards.push({ tx, ty, tz: -200 - Math.random()*360, /* rx, ry, rz */, delay, clipPath });
    }
  }
  return shards;
}
```

關鍵程式碼 — `SplashScreen.css` 用 CSS custom properties 給 keyframe:

```
.splash-shard {
  background-image: url('/favicon.svg');
  /* JSX 內聯 --tx/--ty/--tz/--rx/--ry/--rz */
  transform: translate3d(var(--tx), var(--ty), var(--tz))
    rotateX(var(--rx)) rotateY(var(--ry)) rotateZ(var(--rz));
  animation: shardAssemble 1100ms cubic-bezier(0.22, 1, 0.36, 1) both;
}

@keyframes shardAssemble {
  0% { opacity: 0; transform: translate3d(var(--tx), var(--ty), var(--tz))
    rotate3d(...); }
  15% { opacity: 1; }
  100% { opacity: 1; transform: translate3d(0, 0, 0) rotate3d(0,0,0); }
}
```

簡要說明 / 微調:

- **不引 Three.js:** splash 只跑 < 3 秒, 為一段片頭引 600KB 引擎不划算。CSS 3D transform + clip-path 已能模擬「平面碎裂飛回」的核心視覺。
- **順暢度優化關鍵 4 點:**

1. shard 從 $8 \times 8 = 64$ 降到 $6 \times 6 = 36$ ，動畫層數砍 44%
 2. 拿掉 **filter: drop-shadow**：對 36 個動畫層每幀重算 blur 是最大殺手，改用單層 radial-gradient glow 墊背
 3. 旋轉幅度從 $\pm 540^\circ$ 降到 $\pm 240^\circ$ （每幀位移更小、瀏覽器補幀更平滑）
 4. easing 從 `cubic-bezier(0.16, 1, 0.3, 1)` 改 `(0.22, 1, 0.36, 1)`（中段加速更柔）
- **diagonal-wave delay**：角落（最遠）delay ≈ 0 、中心 delay $\approx 600\text{ms}$ ，視線聚焦的中心最後填滿，符合人類視覺敘事節奏。

Phase 9 — Firebase Hosting 部署

Prompt：最後一個階段，firebase deploy。

做了什麼：建立 `firebase.json` + `.firebaserc`，用 `firebase deploy --only hosting` 部署到 `https://chatroom-72dab.web.app`。

修改的檔案：

- `firebase.json` — 新檔，Hosting-only 設定
- `.firebaserc` — 新檔，綁定 project ID

關鍵程式碼 — `firebase.json`：

```
{
  "hosting": {
    "public": "dist",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "rewrites": [{ "source": "**", "destination": "/index.html" }],
    "headers": [
      { "source": "/index.html", "headers": [
        { "key": "Cache-Control", "value": "no-cache, no-store, must-revalidate" }
      ]},
      { "source": "**/assets/**", "headers": [
        { "key": "Cache-Control", "value": "public, max-age=31536000, immutable" }
      ]}
    ]
  }
}
```

簡要說明 / 微調：

- **SPA rewrites**：所有路徑導去 `/index.html`，react-router 才能正確處理 `/login/chat` 等子路徑。
- **快取策略**：`assets/**` 因 Vite 加 hash 後綴可永久快取；`index.html` 必須 no-cache 否則使用者拿不到新 build 的 hashed asset references。
- **只 deploy hosting**：避免 `firebase deploy` 不小心把空的 `firestore.rules` 蓋掉 console 上的真實規則。
- **bug fix (同 Phase) — firebase init 災難**：實測時 `firebase init` 把 `firebase.json` `public` 從 `"dist"` 改成 `"public"`，並建立 placeholder `public/index.html` + 弱 `firestore.rules` (`allow read,write: if request.auth != null`)。重新部署後線上版顯示 Firebase 預設 welcome 頁。修

復：把 `firebase.json` 改回 `"dist"`、刪 `public/index.html` + 三個 `rules` 檔，重新 `firebase deploy --only hosting`。

3. 備註

- 所有 phase 都是「單一 prompt 啟動 → AI 產生大量檔案 → 我 review + 微調 → npm run build 驗證 → commit」的循環。
- 較複雜的 phase (5、6、8.x) AI 第一次產出後會跑出 bug，會再下幾輪「prompt 描述問題 → AI 提修復 → 我 review」迭代。詳細 bug 與修法見每個 phase 的「簡要說明 / 微調」區塊。
- 整個專案 commit 數約 30+，git log 可看到每個 phase 與後續修正的時序。